TABasco: Task-Aware Browser Tab Optimization using Reinforcement Learning

Team members: **[Insert full names and roll numbers]** | GitHub: github.com/Aarsussh/TABasco
CSL3090 Artificial Intelligence - Assignment 2 | October 2026

Abstract

We formulate browser tab management as a finite-horizon MDP under active-memory pressure and changing tasks. TABasco implements a synthetic session, reward-driven Deep Q-Network (DQN), and LRU, LFU, and random evaluation policies. On 60 held-out seeds, mean return was -48.73 (SD 23.51) for DQN, -59.40 (SD 5.94) for LRU, -59.36 (SD 5.83) for LFU, and -59.13 (SD 5.88) for Random. DQN's invalid-action rate was 0.448; this is a working RL result, not evidence of reliable superiority.

1. Problem and MDP

Browsers keep many tabs available while active pages consume finite memory. Suspending saves active memory but can interrupt a task and require restoration; closing risks lost work and recovery. TABasco models this sequential choice in a simulated browser session.

The state contains N=12 slots and five global features. Each slot has eight normalized values: occupied, active, size/1.35, recency/100, frequency/20, current-task match, estimated revisit probability, and unsaved-work risk. Globals encode active memory/budget, task/3, remaining horizon, elapsed fraction, and occupancy. The fixed observation has 101 float32 values. Tasks range over four contexts. Tab sizes are sampled uniformly from 0.45 to 1.35 synthetic units. Revisit probability changes with age, frequency, and task affinity; sessions include task switches, arrivals, expiration, revisits, and closed-tab history recovery.

The discrete action a=3i+o chooses KEEP, SUSPEND, or CLOSE for slot i. One action advances one time step. Invalid empty/out-of-range actions preserve tabs and incur a penalty. Active tabs use their sampled size; suspended tabs use zero active memory but retain metadata. Budget excess is measured on active memory.

Reward: r=-[1.0 max(0,M-B) + restoration + 1.2 lost-work + 0.45 task-disruption + action-cost]. Ordinary operations cost 0.035; invalid actions cost 0.12. Disruption scales with predicted revisit probability and task match; closing adds unsaved-work loss. Reducing tab count alone has no reward.

2. Agent and experiment

The DQN is a 101-input MLP with two 128-unit ReLU layers and 36 outputs. It uses epsilon-greedy exploration, uniform replay, discount 0.97, Adam at 0.001, batch size 64, Huber TD loss, gradient clipping, and a target network updated every 500 learner updates. Training used CPU for 250 episodes of up to 120 steps, seed 3090. It learns from environment reward and transitions; baselines do not provide labels.

LRU suspends the eligible active tab with oldest last visit; LFU suspends the least accessed (ties by recency); Random uniformly selects an active tab. When none is eligible, each uses its defined fallback action. All policies use seeds 90000-90059 and the same environment parameters; DQN acts greedily. Values below are across 60 one-run-per-seed episodes.

3. Held-out results

Policy	Return mean +/- SD	Mean memory	Invalid rate
DQN	-48.73 +/- 23.51	6.28	0.448
LRU	-59.40 +/- 5.94	1.25	0.006
LFU	-59.36 +/- 5.83	1.30	0.006
Random	-59.13 +/- 5.88	1.36	0.006

DQN averaged 9.45 peak memory against budget 8, with 23.23 violating steps per episode; baselines averaged only 0.55-0.62 violation steps. Their lower returns coincided with higher restoration cost (19.55 LRU versus 6.91 DQN). Useful revisit success was 0.803 DQN and 0.876 LRU. DQN's return SD is large and its invalid actions frequently target empty slots because no action masking is used. Thus the mean return difference is not a robust policy claim.

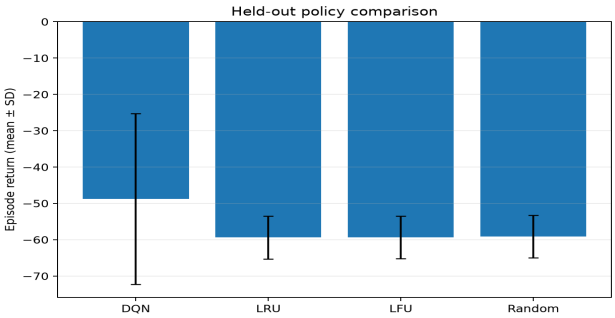


Figure 1. Mean held-out return +/- one standard deviation, 60 shared seeds.

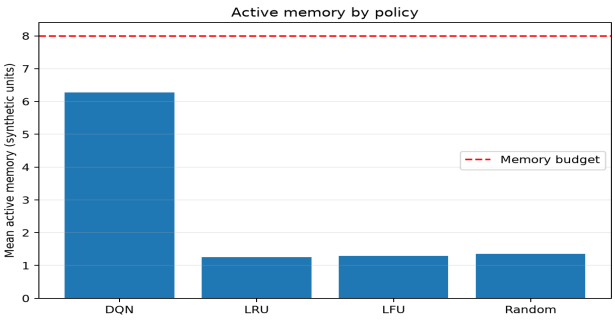


Figure 2. Mean active memory; dashed line marks budget 8.

4. Conclusion and limitations

The project provides a complete reward-driven RL implementation and reproducible experiment pipeline. Synthetic task, memory, and revisit models are simplifications; a single training seed does not establish generalization. Next steps are masking invalid actions consistently, repeated independent training seeds, reward sensitivity analysis, and calibration against browser traces. Complete team names and roll numbers before submission.

5. Training behavior and reproducibility

Training reward is the observed episodic return, including memory overflow, restoration and recovery, lost work, task disruption, and action costs. The curve below plots all 250 logged episode returns with a 20-episode moving mean. Returns remain variable because sessions and revisit/task events are stochastic. Training used seed 3090, and evaluation used the disjoint seed interval 90000-90059. Run arguments and per-episode records are preserved in the repository CSV files.

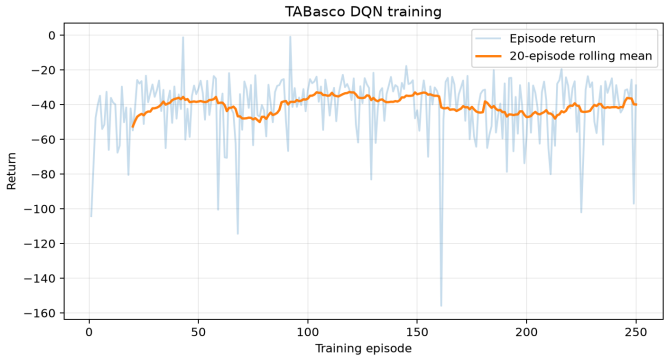


Figure 3. Training episodic return and 20-episode rolling mean from the saved run.

Interpretation

The baselines achieve much lower memory use because they suspend an active tab on almost every eligible decision. That policy creates substantial restoration costs when a page returns. DQN more often leaves tabs active, which reduces restoration costs while allowing memory violations. Its invalid-action rate also wastes decisions and increases action penalties. These tradeoffs explain why a return comparison alone does not identify a uniformly preferable browser policy.

The evaluation summary reports mean and standard deviation across episode seeds, not a confidence interval or an independent retraining estimate. Useful revisit success is the number of useful task-associated revisit events divided by all revisit events; invalid-action rate is invalid decisions divided by decisions. Memory usage is measured in the simulator's relative units. The experiment should be repeated across independent training seeds before making claims about learning stability.

Reproduction commands

```
python -m pip install -r requirements.txt
python -m pytest -q
python train.py --episodes 250 --seed 3090
python evaluate.py --episodes 60 --seed 90000
python plot_results.py
python report/build_report.py
```

The artifact contains a synthetic environment rather than a browser extension. Its return-probability model, risk values, memory sizes, and event rates are explicit assumptions and need calibration with real workload data. Before submission, replace the team member placeholder with the complete names and roll numbers.

References

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529-533, 2015.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.